



UNIVERSITY OF FRONTIER TECHNOLOGY, BANGLADESH
Department of Cyber Security Engineering

Project Report

Assignment Topic : Sentinel-Red - An Automated TTP Emulation Engine

Course title : Object Oriented Programming Language Sessional

Course code : PROG 112

Submitted by : Name : Milton Roy ID : 2404001 Name : MD Tamim Sorder ID : 2404002 Level : 01 Term : 02 Department of Cyber Security Engineering, UFTB	Submitted to: Md Masud Rana Lecturer, Department of Cyber Security Engineering, UFTB
--	--

Date of submission:23 June 2026

Abstract

Sentinel-Red is a C++17 command-line application that automates the controlled emulation of adversary behaviour mapped to the MITRE ATT&CK framework. Rather than manually typing reconnaissance and post-exploitation commands during a security assessment, an operator defines a campaign as a YAML “playbook” — an ordered list of named techniques, each tagged with a MITRE technique ID — and Sentinel-Red executes that campaign against a local machine or a remote SSH-accessible host, records whether each action would likely be flagged by simple detection heuristics, and produces a self-contained HTML report summarising the run.

The project is built around a small set of C++ abstract interfaces (ITechnique, IExecutor, IReporter) so that new attack techniques, new execution targets, and new report formats can be added without modifying existing code. The current implementation (Version 1) supports five technique types — arbitrary shell commands, file modification, network/port enumeration, process enumeration, and user enumeration — over two execution back-ends: a local executor that runs commands via the host shell, and a remote SSH executor built on libssh2 that connects to a target machine over the network and executes commands and file transfers through an encrypted channel. Four ready-made YAML playbooks (Linux reconnaissance, Windows reconnaissance, a custom example, and a detection-logic test) demonstrate the engine end-to-end, and a sample campaign executed against a live host on the local network is included in this report as evidence that the system functions correctly outside of a development environment.

This report documents the system’s design and class architecture, walks through the implementation of each major component with code excerpts, presents the results of an actual campaign run, and discusses the project’s strengths and current limitations. A Version 2.0 upgrade — adding new technique classes, a quantitative detection-scoring engine, a terminal dashboard, and richer YAML-driven target configuration — is outlined as future work.

Introduction

Security teams routinely need to verify that the detective controls they have deployed — antivirus, EDR, SIEM correlation rules, log alerting — actually fire when an attacker performs common post-compromise actions such as listing local users, enumerating running processes, or writing a file to disk. The MITRE ATT&CK framework catalogues these actions as “Tactics, Techniques, and Procedures” (TTPs) and assigns each a stable identifier (for example T1087 for account discovery). Manually reproducing a sequence of TTPs against a test system, one command at a

time, is repetitive and error-prone, and it produces no consistent record of what was run or whether it looked suspicious.

Sentinel-Red addresses this by turning a TTP campaign into data: a YAML file lists the techniques to run, in order, with their MITRE IDs and any parameters they need. A single C++ engine reads that file, builds the corresponding technique objects, executes them against whichever target the operator selected (the local machine, or a remote host over SSH), and writes the outcome of every step into an HTML report. The intent is purely defensive and educational — the tool is meant for authorised testing of systems the operator owns or has explicit permission to test, which is why the program prints a warning banner (“For authorized security testing only”) every time it starts.

Objectives

- Design an extensible, interface-driven C++ architecture in which new attack techniques, execution targets, and report formats can be added as independent classes.
- Implement a YAML-based playbook format so that campaigns can be authored and modified without recompiling the program.
- Support both local execution and remote execution over SSH (via libssh2), including file upload/download.
- Provide simple, transparent detection heuristics that flag technique executions containing suspicious keywords or payload markers, as a stand-in for a real EDR/AV signature.
- Generate a clear, shareable HTML report summarising every technique run, its MITRE ATT&CK reference, its outcome, and whether it was flagged.
- Validate the engine against a real target on the local network, not only in a development sandbox.

System Design and Architecture

Sentinel-Red is built around three small abstract interfaces. This keeps each concern — what a technique does, where it runs, and how results are reported — independent of the others, so any one of them can be extended without touching the rest of the codebase. The whole design follows the classic Strategy / Factory pattern combination taught in object-oriented programming: techniques and executors are interchangeable strategies selected at runtime, and a factory is responsible for turning a YAML-declared type string into the correct concrete object.

Core Interfaces

- **ITechnique** — represents one MITRE ATT&CK technique. Every technique class implements `execute()`, `cleanup()`, and accessor methods for its name, MITRE ID, parameters, result text, and detection flag.
- **IExecutor** — represents “where commands run.” It exposes `connect()`, `disconnect()`, `isConnected()`, `executeCommand()`, `uploadFile()`, and `downloadFile()`. A technique never knows whether it is running locally or over SSH — it only ever talks to an `IExecutor` reference.
- **IReporter** — represents “how results are written up.” It exposes a single `generateReport()` method that receives the full list of executed techniques.

Because every technique depends only on the `IExecutor` interface and not on a concrete executor class, the same `ShellTechnique` object can run unmodified against the local machine or a remote SSH host — this is dependency inversion in practice, not just in theory.

Class Diagram (Textual)

```

ITechnique (abstract)      IExecutor (abstract)
|-- ShellTechnique         |-- LocalExecutor
|-- FileModificationTechnique  |-- RemoteSSHExecutor (libssh2)
|-- NetworkScanTechnique
|-- ProcessEnumTechnique      IReporter (abstract)
|-- UserEnumTechnique         |-- HtmlReporter

TechniqueFactory --creates--> ITechnique subclasses
PlaybookEngine   --owns-->   1 IExecutor, 1 IReporter, N ITechnique
  
```

Figure 1. Simplified class relationships in Sentinel-Red (Version 1).

The Technique Factory

New technique types are registered with `TechniqueFactory` under a string key (e.g. “shell_command”) and a small lambda that knows how to build that technique from a name, MITRE ID, and a parameter map. `PlaybookEngine` never constructs a technique class directly — it asks the factory for whatever type string appeared in the YAML file. This means a new technique class can be added to the project by writing the class itself and adding one registration call, with no changes anywhere else in the engine.

Campaign Execution Flow

1. The operator runs the program with a playbook path and, optionally, remote host credentials.

2. `PlaybookEngine::loadPlaybook()` parses the YAML file with `yaml-cpp`, reading the playbook name, target OS, and the ordered list of techniques, and asks `TechniqueFactory` to build each one.
3. `PlaybookEngine::executeCampaign()` connects the executor (local, or SSH handshake + authentication for remote), then calls `execute()` on each technique in turn, printing a live colour-coded status line per step.
4. Each technique inspects its own output / parameters against a small set of suspicious keywords and sets its own `isDetected()` flag.
5. After the loop, the executor disconnects and `HtmlReporter::generateReport()` is called once, producing a single self-contained HTML file with summary statistics and one card per technique.

Implementation and Code Highlights

This section walks through the implementation of each major component, with excerpts of the actual project source. The full project totals approximately 1,080 lines of C++ across eleven source files and twelve headers.

Component 1: The Technique Interface and the Shell Technique

Every technique implements the same eight-method contract. `ShellTechnique`, the simplest and most-used technique, runs an arbitrary command through the active executor and then scans the command text for a fixed list of suspicious substrings (`mimikatz`, `meterpreter`, `base64 -d`, `reverse-shell` redirections, and similar) to decide whether the action would likely be “detected” by a real defensive product:

```
using namespace std;

const vector<string> ShellTechnique::s_keywords = {
    "mimikatz", "meterpreter", "nc -e", "/dev/tcp", "base64 -d",
    "powershell -enc", "wget http", "curl http", "chmod +s",
    "malicious", "exploit"
};

bool ShellTechnique::execute(IExecutor& executor) {
    if (!executor.isConnected()) { m_result = "Error: not connected"; return false; }
    auto [out, err] = executor.executeCommand(m_cmd);
    m_result = out;
    if (!err.empty()) m_result += "\n[STDERR]\n" + err;

    string lower = m_cmd;
    transform(lower.begin(), lower.end(), lower.begin(), ::tolower);
```

```

m_detected = false;
for (const auto& kw : s_keywords)
    if (lower.find(kw) != string::npos) { m_detected = true; break; }
return true;
}

```

Figure 2. *ShellTechnique::execute* — command dispatch and keyword-based detection scoring.

This demonstrates polymorphism (every technique overrides `execute()` differently), structured exception-free error handling via the connected/not-connected check, and simple string-matching heuristics standing in for a real detection engine.

Component 2: Local and Remote Execution

`LocalExecutor` runs commands through the host shell using `popen()`, capturing stdout directly and redirecting stderr to a temporary file that is read back and deleted:

```

using namespace std;

pair<string,string>
LocalExecutor::executeCommand(const string& cmd) {
    string out, err;
    string tmp = "/tmp/.sr_err_" + to_string(
        reinterpret_cast<uintptr_t>(this));
    string full = cmd + " 2>" + tmp;

    FILE* pipe = popen(full.c_str(), "r");
    if (!pipe) return {"", "popen() failed"};
    array<char,256> buf{};
    while (fgets(buf.data(), buf.size(), pipe)) out += buf.data();
    pclose(pipe);
    // ...read tmp file into err, then remove it
    return {out, err};
}

```

Figure 3. *LocalExecutor::executeCommand* — capturing stdout and stderr separately via `popen()`.

`RemoteSSHExecutor` implements the identical `IExecutor` contract over an authenticated SSH session built with `libssh2`: it opens a raw TCP socket, performs the SSH handshake, authenticates with a username/password, and opens a channel per command to capture stdout and stderr independently:

```

using namespace std;

bool RemoteSSHExecutor::connect() {
    m_sock = socket(AF_INET, SOCK_STREAM, 0);
    sockaddr_in sin{};
}

```

```

sin.sin_family = AF_INET;
sin.sin_port = htons((uint16_t)m_port);
sin.sin_addr.s_addr = inet_addr(m_host.c_str());
::connect(m_sock, (sockaddr*)&sin, sizeof(sin));

m_session = libssh2_session_init();
libssh2_session_handshake(m_session, m_sock);
libssh2_userauth_password(m_session, m_user.c_str(), m_pass.c_str());
m_sftp = libssh2_sftp_init(m_session); // for uploadFile/downloadFile
m_connected = true;
return true;
}

```

Figure 4. *RemoteSSHExecutor::connect* — socket creation, SSH handshake, and password authentication.

File transfer (uploadFile / downloadFile) uses libssh2’s SFTP subsystem in the remote executor and plain std::ifstream/ofstream copying in the local executor, so techniques that need to drop or retrieve a file work identically on either back-end.

Component 3: File Modification, Process, Network, and User Enumeration Techniques

FileModificationTechnique appends operator-supplied content to a target file (simulating a persistence or tampering action) and is flagged as detected if the content contains words such as “malicious”, “payload”, or “exploit”:

```

using namespace std;

bool FileModificationTechnique::execute(IExecutor& executor) {
    string cmd = "echo \"" + m_content + "\" >> " + m_file;
    auto [out, err] = executor.executeCommand(cmd);
    m_result = out.empty() ? "(file written)" : out;

    string lc = m_content;
    transform(lc.begin(), lc.end(), lc.begin(), ::tolower);
    m_detected = lc.find("malicious") != string::npos ||
                 lc.find("payload") != string::npos ||
                 lc.find("exploit") != string::npos;
    return true;
}

```

Figure 5. *FileModificationTechnique::execute* — content-based detection heuristic.

ProcessEnumTechnique and UserEnumTechnique each pick an OS-appropriate command from their “os” parameter (set automatically from the playbook’s target_os field): ps aux / cat /etc/passwd on Linux versus tasklist / net user on Windows. NetworkScanTechnique uses ss or netstat (falling back between the two) to enumerate listening sockets without depending on tools like nmap being present on the target. None of these three currently set isDetected() to true,

reflecting that simple enumeration commands are unlikely to trigger most detection products — a simplification discussed further in the Discussion section.

Component 4: The Playbook Engine and YAML Parsing

PlaybookEngine::loadPlaybook() uses yaml-cpp to read the playbook name, target OS, and the techniques list, injecting the target OS into every technique's parameter map so OS-aware techniques can branch correctly:

```
using namespace std;

for (const auto& node : root["techniques"]) {
    string name  = node["name"].as<string>();
    string mitreId = node["mitre_id"].as<string>();
    string type   = node["type"].as<string>();

    map<string,string> params;
    if (node["parameters"])
        for (const auto& kv : node["parameters"])
            params[kv.first.as<string>()] = kv.second.as<string>();
    params["os"] = m_targetOS;

    auto tech = TechniqueFactory::createTechnique(type, name, mitreId, params);
    if (tech) m_techniques.push_back(move(tech));
}
```

Figure 6. PlaybookEngine::loadPlaybook — turning YAML nodes into technique objects via the factory.

executeCampaign() then connects the executor if needed, runs every technique in sequence while tallying pass/fail/detected counts for the console summary banner, disconnects, and finally invokes the configured reporter.

Component 5: HTML Report Generation

HtmlReporter writes a single dependency-free HTML file with inline CSS using a dark, GitHub-inspired colour scheme. The header block embeds the playbook name, target OS, target host (when running remotely), and a timestamp; a stats row shows total / clean / detected counts and a detection-rate percentage; and one card per technique shows its name, a CLEAN or DETECTED badge, a clickable link to the official MITRE ATT&CK page for its technique ID, and its captured output:

```
using namespace std;

void HtmlReporter::writeTechnique(ofstream& f, const ITechnique& t) const {
    bool det = t.isDetected();
```



```

string mitreUrl = "https://attack.mitre.org/techniques/" + t.getMitreId();
f << "<div class='card ' << (det?"det":"ok") << ">\n"
  << "  <h3>" << esc(t.getName()) << "</h3>\n"
  << "  <span class='badge ' << (det?"det":"ok") << ">"
  << (det?"DETECTED":"CLEAN") << "</span>\n"
  << "  <a class='mitre' href='" << mitreUrl << ">" << t.getMitreId() << "</a>\n"
  << "  <pre>" << esc(t.getResult()) << "</pre>\n</div>\n";
}

```

Figure 7. *HtmlReporter::writeTechnique* — rendering one technique result card, with HTML-escaped output.

All user- and command-derived text is passed through an `HtmlReporter::esc()` helper before being written, escaping `&`, `<`, `>`, and `"` so that command output containing HTML-special characters cannot break the report's markup — a small but important defensive habit when generating HTML from untrusted command output.

Component 6: The Playbooks (YAML Configuration)

Four playbooks ship with the project. `linux_recon.yaml` chains fifteen techniques covering identity, OS/network information, routing, listening sockets, running processes, local accounts, sudo privileges, SUID binaries, world-writable directories, cron jobs, SSH key discovery, environment-variable secret-hunting, and a file-write persistence simulation. `windows_recon.yaml` mirrors this for Windows targets using `whoami`, `systeminfo`, `ipconfig`, `netstat`, `net user`, and similar native commands. `my_custom.yaml` is a minimal three-step template intended as a starting point for new playbooks, and `detection_test.yaml` deliberately exercises both a clean command and a flagged one so the detection logic itself can be sanity-checked:

```

playbook_name: Detection Simulation Test
target_os: linux
techniques:
  - name: Normal System Check
    mitre_id: T1082
    type: shell_command
    parameters:
      command: "uname -a"

  - name: Malicious File Write Simulation
    mitre_id: T1565
    type: file_modification
    parameters:
      file: /tmp/.sr_detection_test
      content: "malicious payload test"

```

Figure 8. *Excerpt of detection_test.yaml* — a playbook authored specifically to validate detection scoring.

Results

The program was compiled with CMake and GCC under C++17, linking against yaml-cpp and libssh2, and tested in both execution modes.

Build Verification

- CMake configuration succeeds and locates yaml-cpp and libssh2 via pkg-config / find_package without manual path edits.
- The project builds as a single executable, sentinel-red, with all eleven source files compiling under -Wall -Wextra with no warnings related to the core technique/executor/reporter logic.
- Local-mode campaigns run correctly with no network configuration required, confirmed against the bundled my_custom.yaml and detection_test.yaml playbooks.

Live Remote Campaign — Windows Reconnaissance

To verify the system beyond a development sandbox, the windows_recon.yaml playbook was executed in remote SSH mode against a real Windows host (192.168.0.201) reachable on the local network. The engine connected over SSH, authenticated, and executed all twelve techniques defined in the playbook before disconnecting and writing the HTML report. The resulting report's summary statistics were:

Metric	Value
Target host	192.168.0.201
Playbook	Windows Full Reconnaissance
Total techniques executed	12
Clean	12
Detected	0
Detection rate	0%

Table 1. Summary statistics from the live Windows reconnaissance campaign, as recorded in sentinel_report.html.

All twelve techniques completed successfully and returned a CLEAN status, which is the expected outcome for this particular playbook — every command in windows_recon.yaml is a legitimate, non-malicious administrative query (whoami /all, systeminfo, ipconfig /all, netstat -ano, net user, net localgroup administrators, and so on), so none of them contain the suspicious keywords that the detection heuristics look for. This confirms two things at once: that the SSH executor can drive a real remote Windows host end-to-end (connect → authenticate → execute twelve distinct

commands → disconnect), and that the detection-scoring logic correctly does not raise false positives on benign reconnaissance commands.

Detection Logic Verification

Running `detection_test.yaml` locally confirmed the opposite case: the “Normal System Check” and “Normal File Listing” steps were marked CLEAN, while “Malicious File Write Simulation” — whose content parameter contains the literal string “malicious payload” — was correctly marked DETECTED, and the `base64-decode` step was flagged due to the `base64 -d` keyword match. This demonstrates that the keyword-matching detection logic behaves correctly on both positive and negative test cases.

Report Output Quality

The generated HTML report renders as a single dark-themed page with no external dependencies (all CSS is inlined), making it easy to share as a standalone file. Each technique card links directly to the corresponding page on attack.mitre.org, so a reviewer can cross-reference the official ATT&CK description for any technique ID without leaving the report. Command output containing special HTML characters was correctly escaped and did not break the page layout.

Overall, the system met its functional objectives: campaigns defined purely in YAML executed correctly against both local and live remote targets, OS-aware techniques selected the correct command for Linux versus Windows, the keyword-based detection heuristic correctly distinguished benign from suspicious actions, and the HTML reporter produced an accurate, readable summary of every run.

Discussion

Strengths

- **Clean separation of concerns.** The `ITechnique` / `IExecutor` / `IReporter` split means the local-vs-remote decision, the choice of technique, and the report format are three independent axes of variation — a textbook demonstration of the Strategy pattern and dependency inversion.
- **Configuration-driven campaigns.** Because playbooks are plain YAML, new test scenarios can be authored or modified by someone who has never touched the C++ source, which is valuable for a tool meant to be used repeatedly across different engagements.
- **Genuine remote capability.** Unlike a toy project that only simulates remote execution, `RemoteSSHExecutor` performs a real SSH handshake, password authentication, command

execution, and SFTP file transfer using libssh2 — confirmed working against an actual host on the network, not just unit-tested in isolation.

- **Self-contained, auditable reporting.** A single HTML file with no external script or stylesheet dependencies, output-escaped for safety, and cross-linked to the authoritative MITRE ATT&CK reference for each technique.
- **Extensibility via the factory pattern.** Adding a new technique type requires writing one class and one registration lambda; the engine, the playbook format, and the reporter require no changes.

Limitations

- **Detection heuristic is a simple keyword match, not a real detection engine.** `isDetected()` is set by scanning command text or file content for a short, hard-coded list of substrings (mimikatz, base64 -d, “malicious”, and similar). This is sufficient to demonstrate the concept and was verified to behave correctly on both the live Windows campaign (correctly all-clean) and the synthetic `detection_test.yaml` playbook (correctly catching the deliberately suspicious steps), but it does not model how a real EDR or SIEM actually scores behaviour — there is no severity weighting, no combination of weak signals, and no scoring based on technique context (for example, the same `network_scan` and `process_enum` techniques never set `isDetected()` at all, regardless of their target).
- **Password-based SSH authentication only.** `RemoteSSHExecutor` authenticates with `libssh2_userauth_password()`; there is no support for key-based authentication, which would be both more secure and more representative of how real engagements are typically configured.
- **No automatic cleanup verification.** Each technique’s `cleanup()` method exists in the interface, and `FileModificationTechnique`’s `cleanup()` does remove the file it wrote, but the engine does not currently call `cleanup()` automatically after a campaign, nor does it verify that cleanup succeeded.
- **Plaintext credentials on the command line.** Remote-mode usage (`sentinel-red playbook.yaml host user pass`) takes the password as a command-line argument, which is visible in shell history and process listings — acceptable for a controlled lab demonstration but not for production use.
- **Limited technique catalogue.** Five technique types cover reconnaissance-style ATT&CK tactics well, but do not yet reach into other tactics such as privilege escalation, lateral movement, or defense evasion beyond the simple file-write persistence simulation.

Lessons Learned

Building Sentinel-Red reinforced several object-oriented programming principles in a practical setting: programming to an interface rather than a concrete class is what made the local-versus-remote execution swap (and, by extension, the planned testability of the whole engine) almost free; a factory that maps configuration strings to constructors is a clean way to let a data file (the YAML playbook) drive runtime behaviour without an explosion of if/else branches; and resource-owning classes that wrap a C API (libssh2, in RemoteSshExecutor) benefit from RAII-style constructors/destructors that guarantee sockets and sessions are cleaned up even on early-return error paths.

Future Work: Version 2.0

Version 1, documented above, is complete and has been verified against a live target. A Version 2.0 upgrade is planned to extend the engine in four directions. This work was in progress at the time of writing — in particular, integrating the yaml-cpp dependency into the updated CMake build was the last open item — and is presented here as the project’s future-work roadmap rather than as completed, tested functionality.

Planned: Additional Technique Classes

New ITechnique implementations are planned to widen ATT&CK tactic coverage beyond reconnaissance — for example techniques modelling credential access, persistence mechanisms beyond a single file write, and basic defense-evasion actions — each following the same pattern already established by ShellTechnique and FileModificationTechnique, and registered through the existing TechniqueFactory with no changes to PlaybookEngine.

Planned: A Quantitative Detection-Scoring Engine

Rather than the current binary CLEAN/DETECTED keyword match, Version 2.0 aims to introduce a weighted scoring model: each technique would contribute a numeric detection score based on multiple signals (command structure, known indicator patterns, technique tactic category) rather than a single substring match, and the campaign-level report would show an aggregate risk score rather than only a flat detection-rate percentage.

Planned: A Terminal (TUI) Dashboard

A text-based user interface is planned to give the operator a live, in-terminal view of campaign progress — a running list of techniques with status indicators updating in place — as an alternative

to the current scrolling console log, for use during interactive engagements where a colleague may be watching the screen.

Planned: YAML-Based Target Configuration

Currently, target connection details (host, user, password, port) are passed as command-line arguments. Version 2.0 plans to move target definitions into YAML as well, alongside the existing playbook format, so that an operator can define a named target once (with its OS, address, and credentials reference) and reuse it across multiple playbook runs without retyping connection details each time.

Outstanding Work at Time of Writing

Implementing the YAML-based target configuration requires the `yaml-cpp` library to be properly linked into the build for the new target-config module, mirroring how it is already linked for playbook parsing. Resolving the `CMakeLists.txt` configuration for this additional `yaml-cpp` usage — alongside verifying the detection-scoring engine and TUI dashboard against a real target, the way Version 1 was verified against the live Windows host described in the Results section — remains the next step before Version 2.0 can be considered complete.

Conclusion

Sentinel-Red demonstrates how object-oriented design principles — abstract interfaces, the Strategy pattern, and a factory for runtime object creation — can be applied to build a small but genuinely functional security-testing tool in C++. Version 1 of the engine successfully loads YAML-defined campaigns, executes five distinct technique types against either a local machine or a real remote host over an authenticated SSH connection, applies a transparent (if simple) detection heuristic to each action, and produces a clear, self-contained HTML report referencing the official MITRE ATT&CK catalogue. The system was not only built but verified against an actual Windows host on the local network, executing a twelve-step reconnaissance playbook end-to-end and producing a correct, accurate report — evidence that the architecture holds up outside of a purely theoretical or sandboxed test.

The project's interface-driven structure leaves clear seams for growth, and the planned Version 2.0 work — additional technique classes, a weighted detection-scoring engine, a terminal dashboard, and YAML-based target configuration — builds directly on those seams rather than requiring a redesign. Taken together, Sentinel-Red met its objective of providing a small, extensible, and practically useful platform for automating authorised TTP emulation, while leaving a well-defined path toward a more capable second version.

References

- MITRE ATT&CK®. Enterprise Matrix and technique reference (attack.mitre.org).
- libssh2 — SSH2 client-side library used for remote execution and SFTP file transfer.
- yaml-cpp — C++ YAML parser/emitter library used for playbook configuration.
- ISO/IEC 14882:2017 — Programming languages — C++ (C++17).
- Bjarne Stroustrup. The C++ Programming Language. Addison-Wesley.
- CMake documentation — cmake.org/documentation.